

W01 — Enterprise RAG: LlamaIndex + Pydantic

Formação AI Data Engineer (AIDE Brasil) — Layer 2

1. OVERVIEW

1.1 O Que Vamos Construir

O W01 transcende o RAG tradicional (“PDF + Vector DB”). Construiremos um **Enterprise DataOps Knowledge Hub**, um sistema multi-fonte containerizado que simula uma infraestrutura de dados corporativa. O sistema ingerirá dados estruturados, semi-estruturados e não-estruturados, utilizando o LlamaIndex para roteamento inteligente e o Pydantic para garantir contratos rígidos de entrada e saída.

O objetivo é transformar o aluno em um *Code-Orchestrator*, capaz de projetar um RAG que dialoga com diferentes paradigmas de banco de dados e os expõe como uma API pronta para produção e consumível por AI Agents.

1.2 Stack Tecnológica

- **LlamaIndex**: Framework de orquestração de dados (Ingestion, Retrieval, Routing).
- **Pydantic**: Contratos de dados e validação de schema.
- **FastAPI**: Camada de serving e API.
- **PostgreSQL**: Ledger (Dados transacionais estruturados).
- **Qdrant**: Memory (Banco vetorial para dados semânticos).
- **Neo4j**: Brain (Banco de grafos para relacionamentos e linhagem).
- **SeaweedFS**: Data Lake S3-compatible (Documentos não-estruturados).
- **Claude Code / MCP**: Agentic Development e consumo final.
- **Docker / Railway**: Ambiente local e deploy em produção.

2. RAG: O PARADIGMA COMPLETO

2.1 O Que é RAG (Visão Enterprise)

Retrieval-Augmented Generation (RAG) não é sinônimo de “Vector Database”. É o padrão arquitetural de **buscar contexto relevante** de fontes externas antes de gerar uma resposta via LLM. A fonte do contexto deve ser ditada pela natureza da pergunta, não pela limitação da ferramenta.

2.2 Evolução do RAG

1. **Naive RAG:** Chunking simples → Embedding → Vector Search.
2. **Advanced RAG:** Pre/Post-retrieval otimizados, re-ranking, query expansion.
3. **Modular RAG:** Módulos intercambiáveis, roteamento de queries.
4. **Agentic RAG (2026):** Agentes autônomos que decidem *quando* buscar, *onde* buscar, avaliam a qualidade do retorno e iteram até encontrar a resposta.

2.3 RAG 2026: Ledger + Memory + Brain

A arquitetura enterprise divide o conhecimento em três domínios cognitivos:

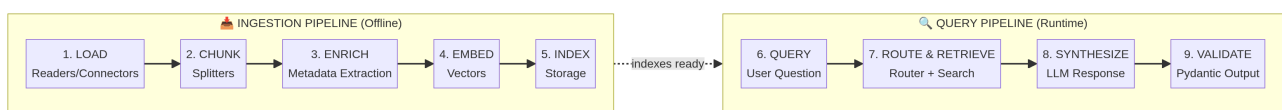
- **Ledger (A Verdade Numérica):** Fatos, transações, métricas. Não usa vetores. Usa Text-to-SQL em bancos relacionais (ex: PostgreSQL).
- **Memory (O Contexto Histórico):** Documentos, logs, políticas. Usa Vector Similarity Search (ex: Qdrant).
- **Brain (As Conexões):** Relacionamentos, dependências, impacto. Usa Graph Traversal (ex: Neo4j).

2.4 Os 9 Steps do RAG Pipeline

O fluxo de vida do dado em um sistema RAG avançado:

Fase de Ingestão (Offline):

1. **Load (Readers/Connectors):** Extração bruta das fontes (Postgres, Mongo, S3) gerando objetos `Document`.
2. **Chunking/Splitting:** Divisão do texto. Técnicas variam de tamanho fixo (`SentenceSplitter`) a semântico (`SemanticSplitterNodeParser`).
3. **Metadata Enrichment:** Uso de LLMs para extrair contexto oculto (ex: `TitleExtractor`, `SummaryExtractor`, `QuestionsAnsweredExtractor`).
4. **Embed:** Conversão de texto em representações vetoriais.
5. **Index & Store:** Armazenamento otimizado (`VectorStoreIndex` no Qdrant, `PropertyGraphIndex` no Neo4j, `SQLDatabase` no Postgres).



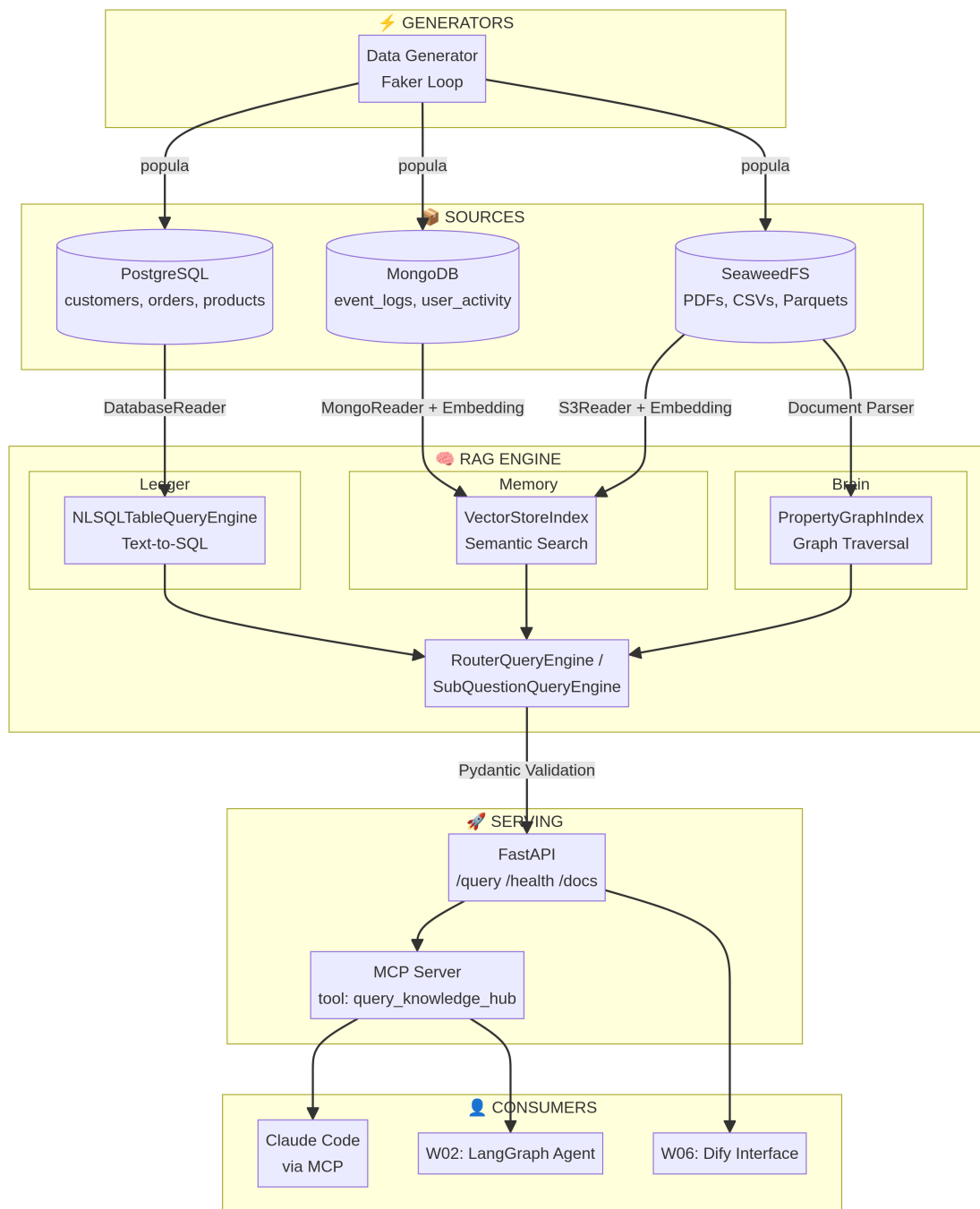
Fase de Consulta (Runtime):

1. **Query:** A pergunta do usuário entra no sistema.
2. **Route & Retrieve:** O sistema decide a melhor fonte (`RouterQueryEngine`) ou divide a pergunta em várias partes (`SubQuestionQueryEngine`) e busca nos índices.
3. **Synthesize:** O LLM combina os fragmentos recuperados em uma resposta coesa (`tree_summarize`, `refine`).
4. **Validate:** O Pydantic força o LLM a retornar a resposta em um schema JSON predefinido, garantindo integração com sistemas downstream.

3. ARQUITETURA DO DATAOPS KNOWLEDGE HUB

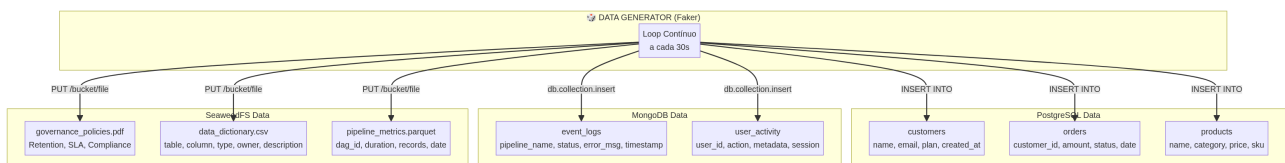
3.1 Visão Geral

O sistema atua como um hub centralizado que responde a perguntas complexas cruzando domínios de dados da empresa.



3.2 Componentes da Infraestrutura

- **Data Generator:** Um script em loop contínuo (Faker) populando ativamente os bancos de dados para simular uma operação viva.



- **Sources:**
 - PostgreSQL: Tabelas `customers`, `orders`, `products`.

- MongoDB: Coleções `event_logs`, `user_activity`.
 - SeaweedFS: Arquivos PDF (manuais), CSV (dicionários).
 - **RAG Engine:**
 - Ledger: LlamaIndex `NLSQLTableQueryEngine` sobre o Postgres.
 - Memory: LlamaIndex `VectorStoreIndex` sobre o Qdrant.
 - Brain: LlamaIndex `PropertyGraphIndex` sobre o Neo4j.
 - **Serving:** FastAPI expando `/query`, envelopado por um MCP Server.
-

4. LLAMAINDEX DEEP DIVE

4.1 O Framework

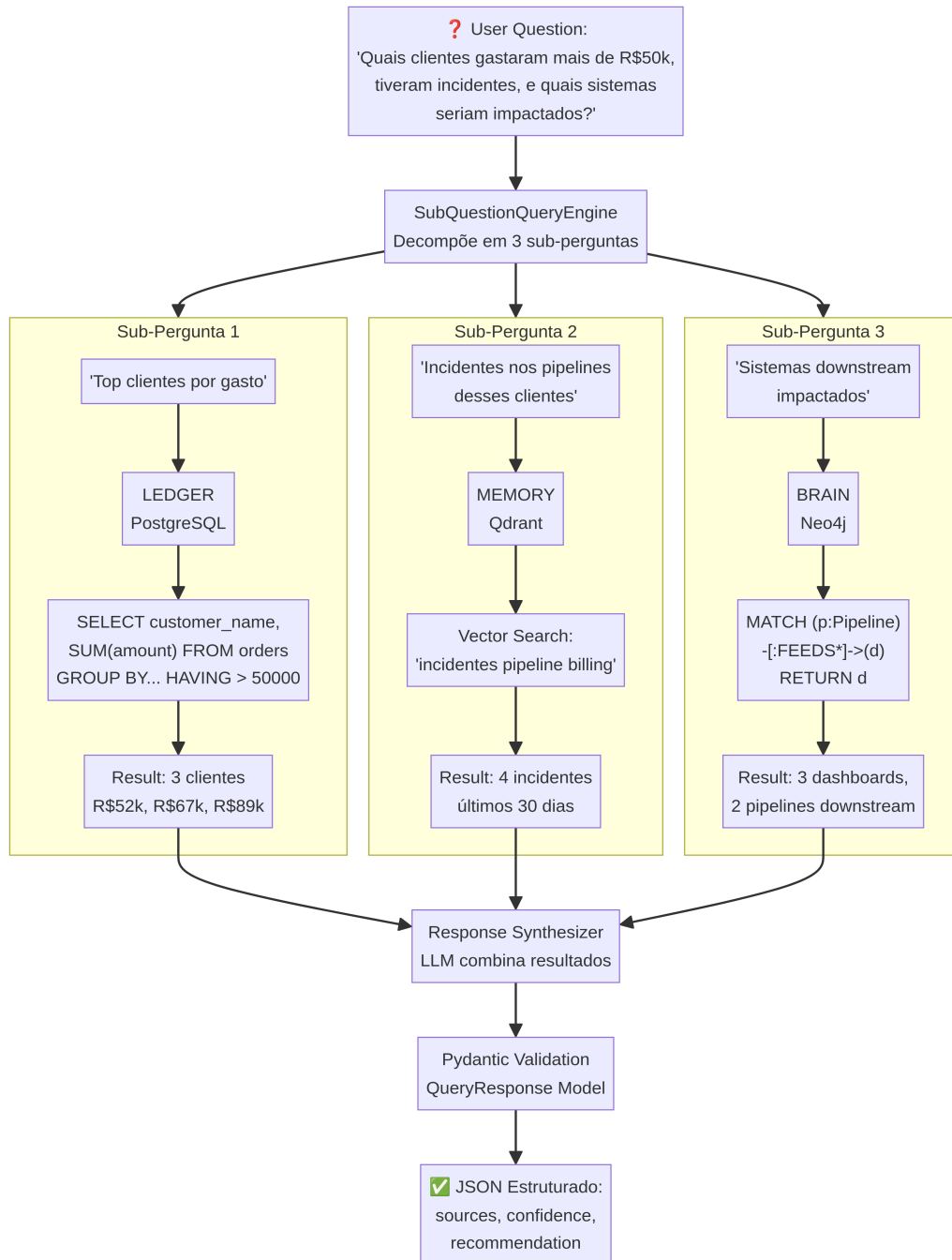
LlamaIndex é o orquestrador. Ele fornece abstrações para conectar dados customizados a LLMs, brilhando em casos de uso multi-fonte e pipelines de ingestão complexos.

4.2 Componentes Chave

- **Readers:** `DatabaseReader` (extrai schemas SQL), `SimpleMongoReader` (extrai JSONs), `S3Reader` (extrai arquivos do SeaweedFS).
- **IngestionPipeline:** Uma cadeia declarativa de transformações. Exemplo: `[SentenceSplitter(), TitleExtractor(), OpenAIEmbedding()]`.
- **Metadata Extraction:** Crucial para melhorar o retrieval. O `QuestionsAnsweredExtractor` pré-gera perguntas que um chunk pode responder, permitindo buscas baseadas em intenção.
- **Query Engines:**
 - `NLSQLTableQueryEngine`: Traduz linguagem natural para SQL, executa no banco e sintetiza a resposta.
 - `RouterQueryEngine`: Usa um LLM para selecionar a melhor *engine* candidata para uma query.
 - `SubQuestionQueryEngine`: Decompõe uma pergunta complexa (“Quais clientes do plano X tiveram incidentes?”) em sub-perguntas roteadas para engines diferentes.

- **Workflows:** Arquitetura orientada a eventos usando decorators `@step`, permitindo controle assíncrono e stateful do fluxo.

Fluxo de Query Cross-Domain



5. CLAUDE CODE CONFIGURATION

5.1 Agentic Development

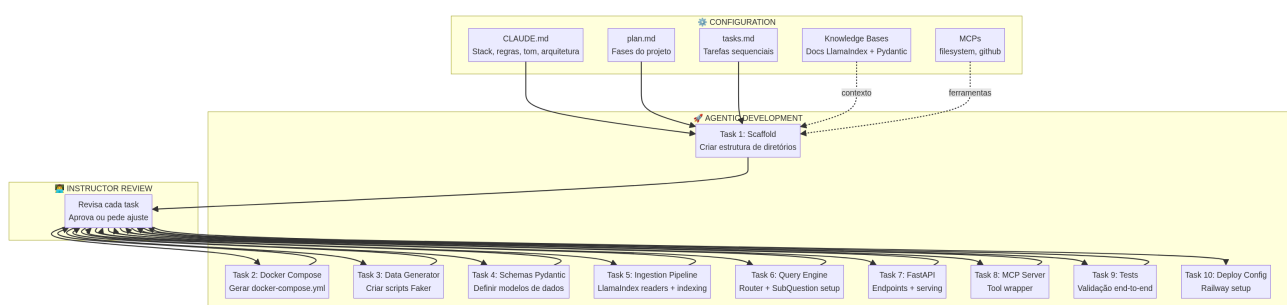
Neste workshop, não escreveremos código linha por linha. Usaremos o Claude Code como nosso executor, guiado por artefatos de controle.

5.2 Artefatos de Configuração

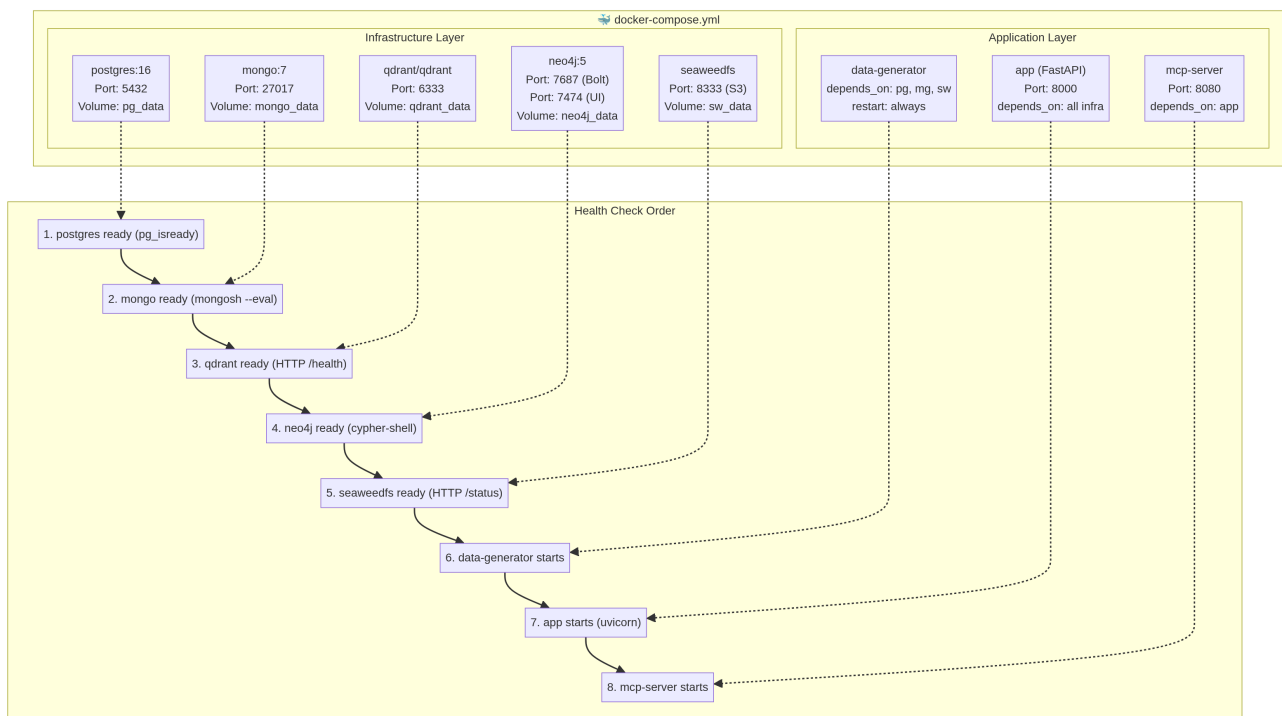
- **CLAUDE.md** : O contrato do projeto. Define a stack, as regras de arquitetura e o tom.
- **plan.md** e **tasks.md** : O mapa de execução. O Claude Code lê as tarefas e as executa sequencialmente.
- **MCPs**: Habilitaremos integrações com o sistema de arquivos e ferramentas locais para dar autonomia ao agente.
- **Knowledge Bases**: Injetaremos a documentação atualizada do LlamaIndex e Pydantic para evitar alucinações com APIs antigas.

6. END-TO-END DEVELOPMENT

6.1 Execução Passo a Passo



1. **Infraestrutura**: O Claude Code gera o `docker-compose.yml` e levanta Postgres, Mongo, Qdrant, Neo4j e SeaweedFS.



1. **Geração de Dados:** Criação do script `data_generator.py` para popular o ambiente continuamente.
2. **Ingestion:** Desenvolvimento do pipeline do LlamaIndex. Extração de schemas do Postgres, inferência de schema do Mongo via Pydantic, e extração de metadados dos PDFs do SeaweedFS.
3. **Query Engine:** Configuração do `SubQuestionQueryEngine` com as ferramentas do Ledger, Memory e Brain.
4. **Validação:** Testes cruzados no terminal.

7. PYDANTIC, FASTAPI, MCP & PRODUCTION DEPLOY

7.1 Pydantic e FastAPI (O Serving)

O RAG precisa ser consumível. O Pydantic define os contratos estritos:

- `QueryRequest` : O que o usuário envia.
- `QueryResponse` : O que o sistema retorna (incluindo fontes, sub-perguntas e resposta sintetizada).

O FastAPI envolve o LlamaIndex, expondo endpoints robustos (`/query` , `/health`).

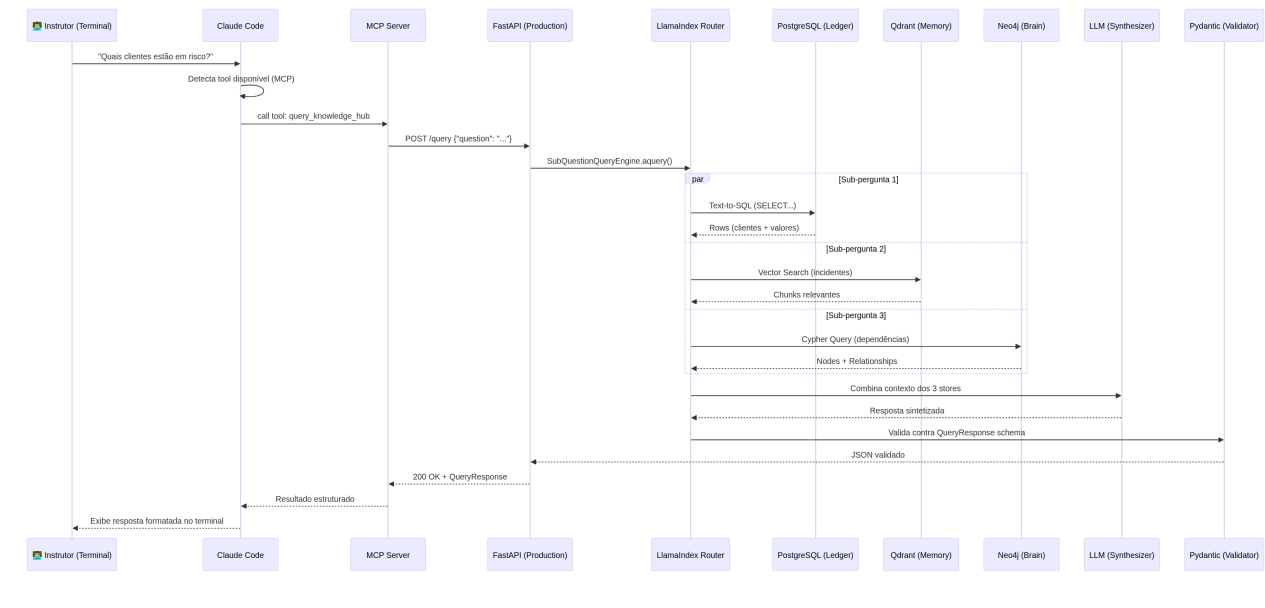
7.2 Model Context Protocol (MCP)

Criamos um MCP Server que expõe a API do FastAPI como uma `tool`. Isso transforma nosso RAG passivo em uma ferramenta ativa no ecossistema de agentes.

7.3 Deploy e O Grand Finale

1. Deploy do Docker Compose via **Railway** (1-click, produção real).
2. O instrutor abre o Claude Code no terminal local.
3. Conecta o Claude Code ao MCP Server recém-deployado.
4. Faz uma pergunta complexa cruzando os três domínios de dados.
5. O Claude Code usa a tool, consulta a API em produção, o LlamaIndex roteia a pergunta, o Pydantic valida, e o resultado perfeito é exibido no terminal.

O AI Agent consome o sistema que ele mesmo construiu.



8. REFERÊNCIAS E PRÓXIMOS PASSOS

8.1 Referências

- [1] LlamaIndex Developers. *Data Connectors (LlamaHub) Documentation*. 2026.
- [2] LlamaIndex. *Introducing Agentic Document Workflows*. Jan 2025.
- [3] AIDE Brasil. *E-book: O AI-Native Engineer*. Layer 0.

8.2 Próximos Passos na Formação

Este Knowledge Hub servirá de base para o restante do Layer 2:

- **W02 (LangGraph):** Transformaremos este sistema reativo em um agente proativo que monitora a qualidade dos dados.
- **W03 (CrewAI):** Multi-agentes documentarão os metadados automaticamente.
- **W06 (Dify):** Criaremos uma interface no-code para usuários de negócio consultarem este RAG.